

Patient Triage System

HI-B-1: Foundations of IT -- Project Work (Topic 1)

1. Project Overview

For this project, I built a terminal-based Python program that acts as a simple patient triage helper for a clinic. The app asks the user for a few basic health vitals -- age, body temperature, pulse, and pain level -- and then automatically figures out which priority level the patient falls into: 'immediate', 'urgent', 'non-urgent', or 'normal'.

2. Program Structure and Class Design

To keep the project clean and modular, I split the program into three separate pieces, so the background logic stays completely away from the terminal prints and inputs:

- **The Patient class:** the main data container that holds all of a patient's vitals together in one object. Right when a Patient object is created, it runs a quick check that automatically cleans up the shortness-of-breath input into a simple "y" or "n", so typos, different capitalisation, or a plain True/False don't mess up the sorting rules later.
- **The TriageSystem class:** handles the actual sorting logic. To keep it completely separated from the user interface, it doesn't contain any print() or input() calls at all -- it just takes a Patient object, checks it against the rules, and returns the final priority category.
- **The interface layer (run_triage_terminal):** runs the live terminal loop. It handles asking the questions, catching invalid input, building the Patient object, and printing the final summary on screen.

I liked this structure because it meant I could test the sorting rules on their own, without needing to fake a whole terminal session just to check one rule.

3. Core Logic and Sorting Rules

The sorting logic uses a step-by-step if/elif/else chain. It checks conditions in order of severity, starting with the most dangerous, life-threatening values, and stops at the first rule that matches:

Category	Condition
Immediate	Shortness of breath present, OR pulse over 130 bpm, OR pulse under 40 bpm.
Urgent	High fever (39.0 C or above), critical hypothermia (35.0 C or below), OR pain 8/10 or higher.
Non-urgent	Pain 2/10 or lower, AND temperature from 36.0 C up to (but not including) 37.5 C, AND age above 12.
Normal	The baseline fallback for standard cases that don't match any of the categories above.

Note on the non-urgent window: the upper bound is exclusive, so a temperature of exactly 37.5 C does not count as non-urgent -- it falls through to normal instead. This matters because it's a boundary the unit tests check explicitly (see Section 5).

4. Input Safeguards and Crash Prevention

To make the application robust and stop it from crashing on bad input, every prompt in the terminal loop is wrapped in validation:

- **Type checks:** every numeric prompt (age, temperature, heart rate, pain) is wrapped in a try/except ValueError block. If someone types letters instead of numbers, the program catches it, prints an error, and asks again instead of crashing.
- **Range checks:** the program rejects impossible values -- a negative age, a pain score outside 0-10, a temperature outside a realistic survivable range (32.0 C to 43.0 C), or a heart rate outside 0-250 bpm.
- **Answer checks:** the shortness-of-breath question and the "next patient?" question only accept yes/no style answers; anything else is rejected with a message and asked again.

5. Testing

The project includes an automated unit test suite (`test_triage_system.py`, using Python's built-in `unittest` module) with 49 tests, covering the `Patient` class's input normalisation, every category in `TriageSystem.evaluate_patient()` including boundary values (e.g. pulse exactly 130 vs. 131, age exactly 12 vs. 13, temperature exactly at 37.5 C), and the terminal loop's input-validation guards, using mocked input/print so no real console interaction is needed. Running the suite confirms all 49 tests pass with 100% line coverage of `triage_system.py`.

6. LLM Use and Reflection

Following the course guidelines to document how AI tools were used on this project:

Where LLMs were used: I don't have a medical background, so I used an LLM to help me understand realistic vital-sign ranges (like what counts as a dangerously high or low pulse, or a concerning fever) so the sorting rules would be based on something reasonable rather than numbers I made up myself. I also used it to help catch syntax errors and small bugs while I was working through the code, to suggest improvements to how the code was written, and for support while setting up and running the test suite.

Adaptation and improvement: I checked the age cutoff used for the non-urgent category (age above 12) against a real reference on how child vs. adult/teen vital signs differ, to make sure the age used to separate the two groups made sense rather than being an arbitrary number. I also ran the unit test suite myself (`python3 -m unittest test_triage_system -v`) and watched all the tests pass before including the results in this report.